Neural Network

Digit Recognition Model

- Dataset: MNIST (28×28 grayscale images)
  - 60k train, 10k test
    - 10 classes
    - 785 cols (1 label + 784 pixels)
- Framework: PyTorch
- Model: MNIST(nn.Module)
  - fc1(784 —>1000)
    - Initial Feature extraction
  - fc2(1000—>500)
    - Feature bottlenecking
  - fc3(500 —>10)
    - Final Classification
  - Output logits
    - Raw Scores
  - ReLU
- Training Process
  - Forward Propagation
    - y_pred = mnist(x_train)
  - Loss Computation
    - F. cross_entropy(y_pred, y_train)
  - Backward Propagation
    - loss. backward()
  - Parameter Update (Optimizer)
    - p -= lr * p.grad
    - mnist.zero_grad()
- Outcomes
  - Test accuracy 89.05%

Flower Classification Model

- Dataset: sklearn's Iris
  - 150 samples
    - 3 targets (setosa, versicolor, virginica)
    - 4 features (sepal/petal length & width)
- Framework: sklearn
- Model: sklearn.linear_model.LogisticRegression(max_iter=1000)
- Training Process
  - LogisticRegression.fit(X,y)
    - Forward Propagation
    - Loss Computation
    - Backward Propagation
    - Parameter Update (Optimizer)
- Outcomes
  - Test accuracy=0.973
  -

    Two features

House Price Prediction Model

- Dataset: California Housing
  - 20,640 samples (20% test sets)
    - target: median house value
    - 8 features (MedInc, HouseAge, AveRooms, AveBedrms, Population, AveOccup, Latitude, Longitude)
      - MedInc: Median income in the block group
      - HouseAge: Median house age in the block group
      - AveRooms: Average number of rooms per household
      - AveBedrms: Average number of bedrooms per household
      - Population: Total population of the block group
      - AveOccup: Average number of household members
      - Latitude: Latitude of the block group
      - Longitude: Longitude of the block group
- Framework: sklearn
- Model: MLPRegressor
  - Training hyperparameters
    - hidden_layer_sizes=(100, 50)
    - learning_rate_init=0.001
    - activation='relu'
    - solver='adam'
    - max_iter=1000
- Training Process
  - mlp.fit(x_train, y_train)
    - Forward Propagation
    - Loss Computation
    - Backward Propagation
    - Parameter Update (Optimizer)
  - loss curve
    -

      - Quadratic nature of the MSE loss
- Outcomes
  - MAE=0.626, RMSE=0.86 (× $100,000)

Stock Price Prediction Model

- Dataset: prices-split-adjusted.csv
  - 851264 samples (10% test sets, 10% validation sets)
    - 851 rows, 501 stocks
    - columns: symbol, open, close, low, high, volume
      - ```
        symbol,open,close,low,high,volume
        0,AAL,43.75,43.65,43.26,43.79,11475900
        1,AAL,43.97,43.82,43.62,44.07,8462700
        2,AAL,44.25,44.29,43.91,44.41,7732700
        3,AAL,44.265,44.095,43.94,44.37,8271100
        4,AAL,44.19,44.41,44.09,44.54,5202900
        5,AAL,44.16,43.77,43.69,44.16,6304300
        6,AAL,43.62,43.57,43.36,43.77,7775400
        7,AAL,43.66,43.96,43.51,44.06,6651000
        8,AAL,43.99,44.062,43.77,44.132,5405900
        9,AAL,44.01,43.75,43.571,44.062,5274900
        ```
      - Need Normalization
  - Manipulate data
    - Choose a specific stock
      - Drop feature: symbol & volume
      - Use MinMaxScaler to normalize stock data
        - Split dataset with seq_len = 20
      - Huge Scale Difference
- Framework: TensorFlow
- Training hyperparameters
  - n_steps = seq_len - 1
  - n_inputs = 4
    - open, high, low, close
  - n_neurons = 200
    - hidden state
  - n_outputs = 4
    - open, high, low, close
  - learning_rate = 0.001
  - batch_size = 50
    - 50/1394
  - n_epochs = 100
  - optimizer=Adam
- Model: RNN+LSTM
  - Basic RNN
    - Input(n_steps, n_inputs)
    - SimpleRNN(200, return_sequences=False)
      - (50, 19, 4)
    - Dense(4)
      - (50, 200)
  - RNN LSTM
    - Input(n_steps, n_inputs)
      - (50, 19, 4)
    - LSTM(200, return_sequences=True)
      - (50, 19, 200)
    - LSTM(200, return_sequences=False)
      - p (50, 4)
    - Dense(4)
      - (50, 200)
- Training Process
  - Basic RNN
    - model.fit(x_train, y_train, batch_size=50, epochs=100, validation_data=(x_valid, y_valid))
      - Train (Each iteration)
        - Forward Propagation
        - Loss Computation
          - 0.05 -> -8e-05
        - Backward Propagation
        - Parameter Update (Optimizer)
      - Validate (Each epoch)
        - Forward (x_valid)
        - val_loss / val_mae Calculation
          - loss generally trends downward -> 2.5e-04
    - Train Loss = -8.6e-05
    - Train MAE = -0.0068
    - Val Loss = -2.48e-04
    - Val MAE = -0.0108 (=)
  - RNN+LSTM
    - model.fit(x_train, y_train, batch_size=50, epochs=100, validation_data=(x_valid, y_valid))
      - Train (Each iteration)
        - Forward Propagation
        - Loss Computation
          - 0.03 -> -6.3e-05
        - Backward Propagation
        - Parameter Update (Optimizer)
      - Validate (Each epoch)
        - Forward (x_valid)
        - val_loss / val_mae Calculation
          - loss generally trends downward -> 2.1e-04
    - Train Loss = -6.3e-05 (Better)
    - Train MAE = -0.0055 (Better)
    - Val Loss = -2.10e-04 (Better)
    - Val MAE = -0.0108 (=)
    - Better fit
- Outcomes
  - Basic RNN
    - Test MAE = 0.0095 (Better)
    - Test MSE = 0.0002 (=)
  - RNN+LSTM
    - Test MAE = -0.0113
      - Little over fit
    - Test MSE = 0.0002 (=)

Image Classification Model

- Dataset: Modified Fashion-MNIST (28×28 grayscale images) (https://zenodo.org/abs/7006.0747)
  - 55k train, 5k val, 10k test
    - 10 classes
      - T-shirt, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot
    -

      Example
  - Data Normalization
    - x/255
- Framework: Keras
- Model: Keras Sequential (CNN)
  - (batch, 28, 28, 1)
  - Structure
    - ```
      Model: "sequential_2"
      _________________________________________________________________
      Layer (type)                 Output Shape              Param #
      =================================================================
      conv2d_6 (Conv2D)            (batch, 26, 26, 32)       320
      max_pooling2d_4 (MaxPooling2D)  (batch, 13, 13, 32)    0
      dropout_4 (Dropout)          (batch, 13, 13, 32)       0
      conv2d_7 (Conv2D)            (batch, 11, 11, 64)       18,496
      max_pooling2d_5 (MaxPooling2D)  (batch, 5, 5, 64)      0
      dropout_5 (Dropout)          (batch, 5, 5, 64)         0
      flatten_2 (Flatten)          (batch, 1600)             0
      dense_4 (Dense)              (batch, 128)              204,928
      dropout_6 (Dropout)          (batch, 128)              0
      dense_5 (Dense)              (batch, 10)               1,290
      =================================================================
      ```
  - Pipeline Visualization
    - Conv2D
      - Apply learnable filters over the input to extract local patterns (e.g. edges, textures) and produce feature maps.
    - MaxPooling2D
      - Downsample feature maps by taking the maximum in each sliding window, reducing spatial size and computation.
    - Dropout
      - Randomly set a fraction of units to zero during training to reduce overfitting
    - Flatten
      - Reshape the multi-dimensional feature maps into a single 1D vector for feeding into fully connected layers.
    - Dense
      - Fully connect every input to every output neuron to combine features and produce class scores or predictions.
    -

      - Visualized with netron
  - Number of parameters
    - ```
      Total params: 422,778 (1.57 MB)
      Trainable params: 422,778 (1.57 MB)
      Non-trainable params: 0 (0.00 B)
      ```
- Training Process
  - model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    - loss='categorical_crossentropy'
      - -Σ y_true_i * logy_pred_i)
    - metrics=['accuracy']
      - Number of correctly-predicted Samples/ Total Number
  - model.fit(x_train, y_train, batch_size=64, epochs=10, validation_data=(x_valid, x_valid), callbacks=[checkpointer])
    - Train (Each iteration)
      - Forward Propagation
      - Loss Computation
        - 0.8560 -> 0.2715
      - Backward Propagation
      - Parameter Update (Optimizer)
    - Validate (Each epoch)
      - Forward (x_valid)
      - val_loss / val_mae Calculation
        - 0.1908 ->0.2267
    - Save Best Only >> model.weights.best.h5
- Outcomes
  - Test accuracy: 0.9102
  -
    Visualization (matplotlib)

Object Detection Model

- Dataset (Image+Label)
  - Test set: 220 (Not annotated)
  - Train set: 1863
  - Validation set: 200
    - nc = 10
      - person
      - cat
      - dog
      - bottle
      - banana
      - orange
      - chair
      - laptop
      - book
      - backpack
    - Annotated with Label Studio
- Framework: Ultralytics YOLO0
- Model: YOLO11n
  - ~2.6M Arguments
  - Backbone
    - Conv (Downsampling)
      - Reduce feature map resolution and expands the receptive field
        - P1/2
        - P2/4
        - P3/8 (High Res)
        - P4/16 (Mid Res)
        - P5/32 (Low Res)
          - Spatial detail
          - Object level
            - Multi scale Feature Maps
    - C3k2 Block
      - Deep feature extraction
    - C2PSA (C2F with PSA)
      - Salient region focusing
    - SPPF (Global)
      - Global Context Awareness
  - Head
    - FPN (Structural Pattern)
      - Upsample + Concat
        - Aggregate semantic & spatial info
      - C3k2 (Refining)
        - Feature Smoothing (Eliminating fusion noise)
    - Detect(nc=10)
      - Decoupled Output
        - Result: 10-class object detection with confidence scores
- GPU: RTX 4070
- OS: Windows 11
- Training hyperparameters
  - 50 epochs
  - patience = 10
    - Stopped early when epochs < 35
  - batch = 16
  - Optimizer: Adam
    - Faster and more friendly to small data sets
  - lr0 = 0.001
- Outcomes
  -

- Limit
  - Core Bottleneck: Severe False Negatives (Miss Rate)
    - epochs++
  - Semantic Confusion & Feature Overlap
    - Hard Example Mining
  - Background Hallucinations (False Positives)
    - negative samples (unlabeled background images)
  - Diagonal accuracy (True Positives) remains below 0.50 for most classes
    - witch to the AdamW

Presented with xmind
      ```